

Rapport Final de Projet

Site d'Arcade Rétro Multijoueur

Membres de l'équipe :

ELLAIK Fadwa

OUNCHARI Imane

BENABOUD Mehdi

EL MOUBAKKIR Mouna

Table des matières

1	Introduction et Vision du Projet	3
1.1	Présentation générale	3
1.2	Objectifs fonctionnels	3
1.3	Évolution par rapport aux itérations précédentes	4
2	Architecture Technique	5
2.1	Vue d'ensemble	5
2.2	Structure du projet	5
2.2.1	Backend (Spring Boot)	5
2.2.2	Frontend (React)	6
2.3	Base de données	6
2.3.1	Migration MySQL vers Spring Data JPA	6
2.3.2	Modèle de données	6
3	Communication Frontend – Backend – Base de Données	7
3.1	API REST (HTTP/Axios)	7
3.2	WebSockets – synchronisation temps réel	7
3.2.1	Configuration serveur	7
3.2.2	Gestion de l'état côté client	7
4	Système d'Invitation et Déroulement d'une Partie	8
4.1	Flux d'invitation complet	8
4.2	Déroulement d'une partie	8
4.3	Fin de partie et enregistrement du score	9
5	Présentation des Mini-Jeux	10
5.1	Jeux implémentés	10
5.2	Architecture commune des jeux	10
6	Authentification et Sécurité	11
6.1	JWT (JSON Web Token)	11

7	Limites et Problèmes Rencontrés	12
7.1	Asteroid Duel – jeu non fonctionnel en multijoueur	12
7.2	Autres limitations	12
8	Conclusion	13
8.1	Bilan du projet	13
8.2	Perspectives d'amélioration	13
A	Architecture du Projet	14
B	Diagramme de Séquence – Invitation et Lancement de Match	15
C	Captures d'écran de l'interface	16

Chapitre 1

Introduction et Vision du Projet

1.1 Présentation générale

Ce projet a pour objectif de développer une plateforme web d'arcade rétro permettant à plusieurs joueurs, connectés depuis des machines différentes sur le même réseau local, de s'affronter en temps réel dans une sélection de mini-jeux. L'interface s'inspire de l'esthétique des bornes d'arcade des années 70–80 : pixel art, palette de couleurs contrastée, typographies rétro et animations soignées.

1.2 Objectifs fonctionnels

- **Gestion des comptes** : inscription, connexion sécurisée (JWT), profil et historique de scores.
- **Catalogue de jeux** : présentation des mini-jeux disponibles avec vignette et description.
- **Système d'invitation** : un joueur invite un autre par pseudo ; l'invité accepte ou décline.
- **Partie multijoueur en temps réel** : synchronisation de l'état du jeu via WebSockets (STOMP/SockJS).
- **Tableau des scores (Leaderboard)** : classement par jeu, mis à jour après chaque partie.

1.3 Évolution par rapport aux itérations précédentes

Itération	Travail réalisé
1	Spécifications, diagrammes UML (cas d'utilisation, classes, séquence), premières classes Java.
2	Architecture en couches Spring Boot : Repository, Service, Controller pour l'entité <i>Jeu</i> .
Finale	Passage de MySQL brut à Spring Data JPA + JDBC, ajout des WebSockets, implémentation complète de tous les jeux et du système d'invitation.

Chapitre 2

Architecture Technique

2.1 Vue d'ensemble

L'application repose sur une architecture client-serveur classique à trois couches, enrichie par un canal bidirectionnel WebSocket pour la synchronisation en temps réel des parties.

Pile technologique	
Couche	Technologie
Frontend	React 18, React Router, Axios, STOMP.js, SockJS-client
Backend	Java 17, Spring Boot 3, Spring Data JPA, Spring WebSocket
Base de données	MySQL 8 (accès via JDBC / Hibernate)
Authentification	JWT (jjwt)
Communication temps réel	WebSocket + protocole STOMP via SockJS

2.2 Structure du projet

L'organisation du code source suit les bonnes pratiques des frameworks utilisés, garantissant une séparation claire des responsabilités.

2.2.1 Backend (Spring Boot)

L'architecture logicielle côté serveur est organisée en couches classiques : configuration, contrôleurs (REST et WebSocket), services métier, accès aux données (repositories), modèles d'entités et objets de transfert de données (DTO).

`com.arcade`

config/	(Configurations de sécurité et WebSockets)
controller/	(Points d'entrée REST et STOMP)
service/	(Logique métier et traitements)
repository/	(Interfaces Spring Data JPA)
model/	(Entités mappées en base de données)
dto/	(Objets de transfert de données)

2.2.2 Frontend (React)

L'interface utilisateur est structurée pour favoriser la réutilisabilité des composants, avec une séparation entre les pages principales, les services d'appel API et la logique des jeux.

src/	
services/	(Configuration Axios et endpoints)
hooks/	(Hooks personnalisés comme useMultiplayerSync)
pages/	
games/	(Composants spécifiques à chaque mini-jeu)
...	(Pages générales : Login, Home, Lobby, Profile)

2.3 Base de données

2.3.1 Migration MySQL vers Spring Data JPA

Lors de l'itération 2, l'accès à la base de données était manuel. En itération finale, le projet s'appuie intégralement sur **Spring Data JPA** couplé au pilote MySQL JDBC. Hibernate est configuré pour générer et mettre à jour automatiquement le schéma de la base de données à partir des entités Java annotées, simplifiant ainsi grandement la persistance des données.

2.3.2 Modèle de données

Les principales entités implémentées et relationnées en base sont les suivantes :

Entité	Attributs clés
Utilisateur	id, pseudo (unique), email, motDePasse
Jeu	id, titre, description, urlImage
Score	id, valeur, date, @ManyToOne Utilisateur, @ManyToOne Jeu
Match	id, @ManyToOne Jeu, player1, player2, status, winnerId, timestamps
Invitation	id, sender, receiver, @ManyToOne Jeu, status, timestamps

Chapitre 3

Communication Frontend – Backend – Base de Données

3.1 API REST (HTTP/Axios)

Le frontend communique avec le backend via une API RESTful. Afin de sécuriser les échanges, un intercepteur Axios a été mis en place pour injecter automatiquement le token d'authentification JWT dans les en-têtes (Headers) de chaque requête HTTP sortante.

3.2 WebSockets – synchronisation temps réel

Pour la synchronisation des parties en cours et les notifications d'invitation, l'application utilise une communication bidirectionnelle via **STOMP over SockJS**.

3.2.1 Configuration serveur

Le serveur Spring Boot est configuré pour exposer un point de terminaison WebSocket. Un "Message Broker" gère la distribution des messages : les clients envoient leurs actions vers des routes spécifiques (préfixées par `/app`), et le serveur diffuse les mises à jour sur des canaux d'abonnement (préfixés par `/topic`).

3.2.2 Gestion de l'état côté client

Côté React, un hook personnalisé a été développé pour abstraire la complexité de la connexion WebSocket. Il se charge d'établir la connexion avec le token JWT, de s'abonner au canal spécifique au match en cours, d'écouter les événements envoyés par l'adversaire (en filtrant ses propres messages), et de fournir une fonction simple pour envoyer ses propres actions de jeu au serveur.

Chapitre 4

Système d'Invitation et Déroulement d'une Partie

4.1 Flux d'invitation complet

Le processus d'invitation s'appuie à la fois sur l'API REST pour la persistance et sur les WebSockets pour la réactivité.

1. **Envoi de l'invitation** : le joueur A saisit le pseudo du joueur B et choisit un jeu dans le Lobby via une requête POST classique.
2. **Notification WebSocket** : le serveur publie un message sur le canal d'invitation personnel du joueur B. Son interface se met à jour instantanément sans rechargement de page.
3. **Acceptation** : le joueur B accepte l'invitation, déclenchant la création officielle d'un objet **Match** en base de données.
4. **Redirection simultanée** : le serveur envoie un signal de démarrage aux deux joueurs via WebSocket, ce qui déclenche leur redirection synchronisée vers le plateau de jeu.

4.2 Déroulement d'une partie

Une fois redirigés, les deux joueurs chargent la même page de jeu. Le processus suit un modèle **publish–subscribe** :

1. Chaque joueur interroge l'API pour déterminer son rôle (joueur 1 ou joueur 2).
2. Lorsqu'un joueur effectue une action valide (ex : placer un pion), l'action est appliquée localement puis envoyée au serveur via WebSocket.
3. Le serveur redistribue instantanément cette action sur le canal du match.
4. Le client de l'adversaire reçoit l'action et met à jour son interface en conséquence.

4.3 Fin de partie et enregistrement du score

Lorsque la logique locale d'un des clients détecte une condition de victoire, d'égalité ou de défaite, le résultat est envoyé au backend via une requête REST sécurisée. Le serveur persiste le score final en base de données et met à jour le classement global (Leaderboard).

Chapitre 5

Présentation des Mini-Jeux

5.1 Jeux implémentés

Jeu	Description
Tic-Tac-Toe	Morpion classique 3×3, deux joueurs (X et O).
Connect 4	Puissance 4, grille 6×7, pions rouge/jaune.
Battleship	Bataille navale, chaque joueur place ses bateaux puis attaque tour à tour.
Brawlbots	Combat de robots au tour par tour : chaque joueur choisit une action (attaque, bouclier, soin) qui est résolue simultanément.
Match Masters	Mini-jeu de score cumulatif : les joueurs s'affrontent sur plusieurs manches, le plus haut score total gagne.

5.2 Architecture commune des jeux

Tous les jeux partagent la même fondation technique. Chaque composant React de jeu est responsable du maintien de son état local (grille, points de vie, tour actuel) tout en déléguant la synchronisation réseau au hook STOMP développé de manière générique pour l'application. Cette mutualisation du code réseau a permis d'accélérer le développement des différents mini-jeux.

Chapitre 6

Authentification et Sécurité

6.1 JWT (JSON Web Token)

L'application sécurise ses routes API grâce au standard JWT. À la connexion, le backend vérifie les identifiants et génère un token signé contenant les informations essentielles de l'utilisateur. Ce token est conservé de manière sécurisée côté client et transmis systématiquement pour prouver l'identité de l'utilisateur lors de ses actions, y compris lors de l'établissement de la connexion WebSocket.

Chapitre 7

Limites et Problèmes Rencontrés

7.1 Asteroid Duel – jeu non fonctionnel en multijoueur

Le jeu *Asteroid Duel* a été développé mais n'a pas pu être rendu opérationnel en mode multijoueur sur machines distinctes.

Cause technique : ce type de jeu d'action nécessite une synchronisation à haute fréquence (plusieurs dizaines de trames par seconde). Notre architecture WebSocket STOMP, parfaitement adaptée pour des jeux au tour par tour, introduit une latence incompatible avec les exigences temps réel de ce type de jeu (gestion des collisions fluides, positionnement exact des vaisseaux).

Pour résoudre ce problème, il faudrait implémenter un serveur dédié intégrant une boucle de jeu (*game loop*) avec des algorithmes d'interpolation côté client, ce qui représentait une complexité débordant du cadre temporel de ce projet.

7.2 Autres limitations

- Le jeu a été testé et optimisé pour un fonctionnement en **réseau local**. Un déploiement public nécessiterait une configuration réseau supplémentaire (pare-feu, proxys inverses).
- La gestion de la déconnexion inattendue d'un joueur en pleine partie n'est pas complètement gérée (la partie peut rester bloquée en l'état).
- La validation en profondeur des tokens d'authentification sur chaque message WebSocket pourrait être renforcée côté serveur.

Chapitre 8

Conclusion

8.1 Bilan du projet

Ce projet a permis de mettre en pratique l'ensemble des compétences acquises au cours de la formation :

- **Conception** : modélisation UML (cas d'utilisation, classes, séquence).
- **Backend** : développement d'une API REST complète avec Spring Boot, Spring Data JPA et MySQL.
- **Temps réel** : intégration des WebSockets (STOMP/SockJS) pour la synchronisation multijoueur et les événements asynchrones.
- **Frontend** : conception d'une Single Page Application (SPA) en React avec gestion fine des états et du routage.
- **Travail d'équipe** : développement par itérations, gestion des versions et répartition logique des tâches.

Le résultat est une plateforme fonctionnelle offrant cinq jeux complets jouables en réseau local, supportée par un système de profils, d'invitations dynamiques et de classements compétitifs.

8.2 Perspectives d'amélioration

- Déploiement automatisé sur un serveur distant (via Docker).
- Implémentation d'une gestion robuste des déconnexions avec forfaits automatiques ou reconnexion.
- Ajout de fonctionnalités sociales complémentaires : système d'amis, mode spectateur et chat global.

Annexe A

Architecture du Projet

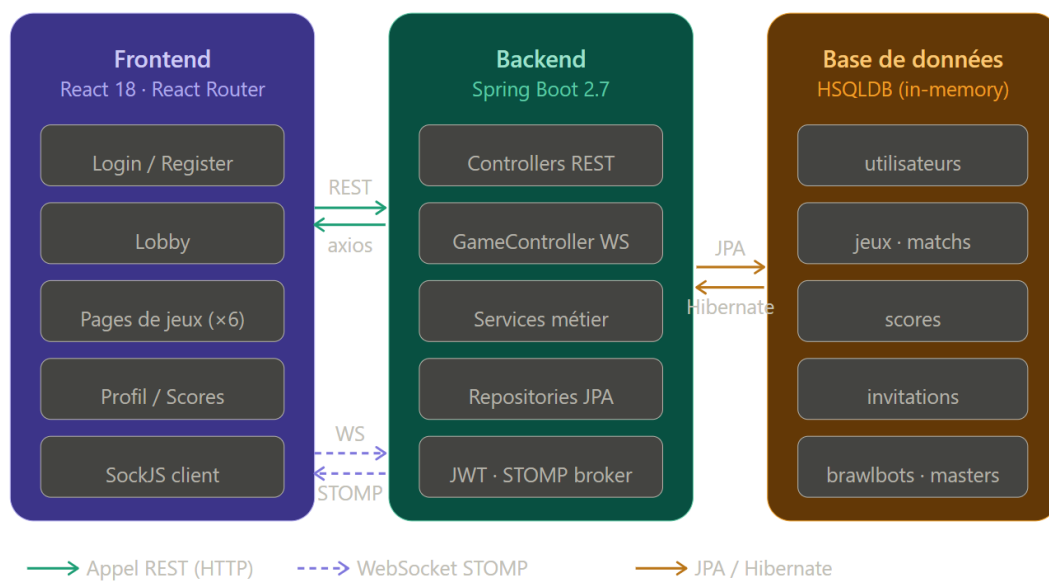


FIGURE A.1 – Architecture du projet

Annexe B

Diagramme de Séquence – Invitation et Lancement de Match

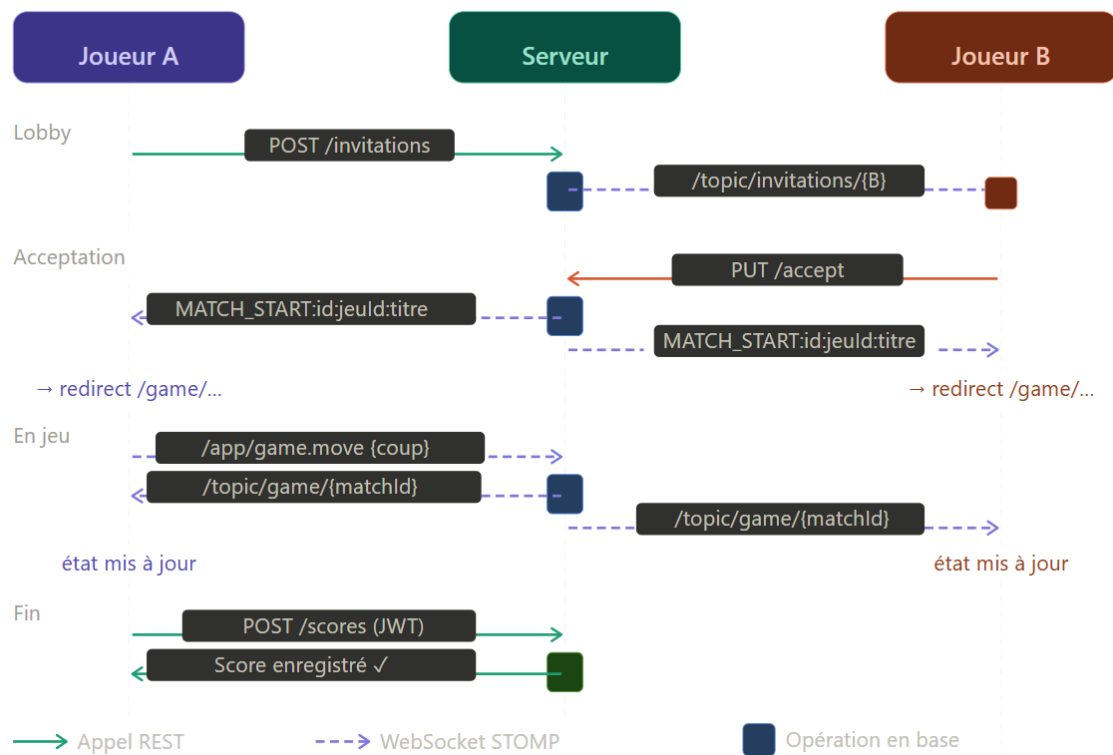


FIGURE B.1 – Diagramme de séquence

Annexe C

Captures d'écran de l'interface



FIGURE C.1 – Page Accueil – liste des jeux et système d'invitation

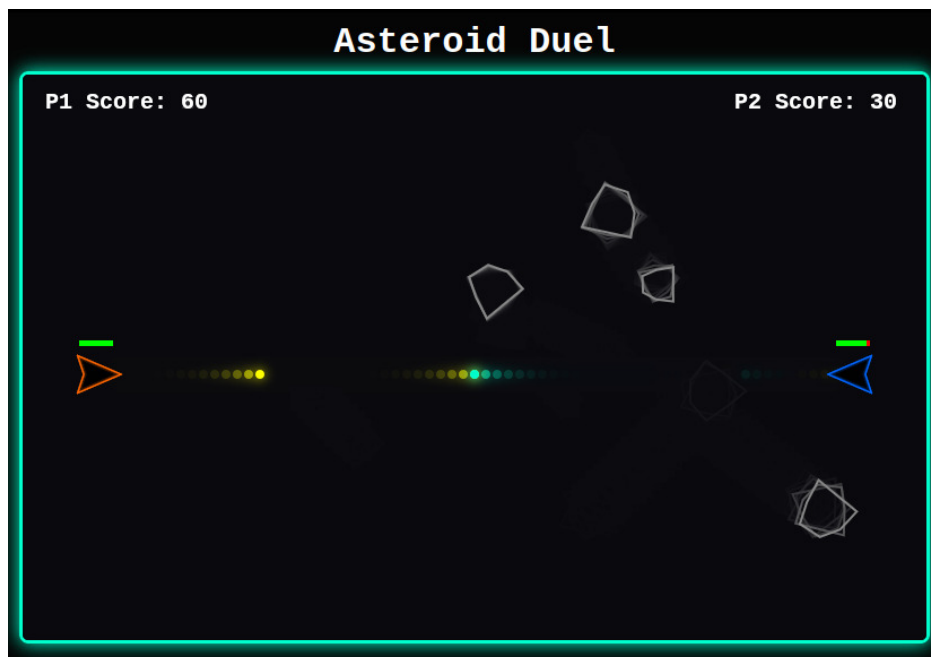


FIGURE C.2 – Jeu Asteroid



FIGURE C.3 – Jeu Bataille navale